Interactive Terminal Execution Feedback for Repository-Level Test Generation: An Empirical Study on TestExplora

Chi Tam Le, Tran Gia Huy Le, Tien Phu Nguyen, Van An Truong
FPT University, Vietnam

***Abstract*—Automated software testing is central to software quality assurance, yet it remains costly: in industrial settings, testing can account for up to 50% of a project's development budget. Static language models used for test generation exhibit a compliance bias—they tend to accept erroneous output rather than identifying the underlying logic flaw. The effect of real-time interactive terminal execution feedback on repository-level test suite generation, and its potential to mitigate this bias, has not previously been studied. This article introduces a dual-agent, open-source system that pairs DeepSeek-V3, operating through the SWE-agent CLI bash terminal, with Llama-3.3-70B. We evaluated the approach on 1,552 Python tasks from the TestExplora benchmark. The system achieved an average Fail-to-Pass rate of 35.05% at Pass@1, more than double the static baseline's 16.60%. At Pass@3, this rate reached 71.59% (Wilcoxon one-tailed test, $p = 0.000000 < 0.05$; Cliff's $\delta = 0.3018$, a medium effect size). These results indicate that incorporating real-time execution feedback reduces compliance bias in static LLMs and supports the feasibility of deploying proactive automated testing systems in CI/CD pipelines.**

***Index Terms*—large language models, unit test generation, interactive terminal feedback, compliance bias, empirical study**

I. Introduction

Automated software testing plays a pivotal role in modern software quality assurance, enabling continuous integration pipelines to validate system behavior across complex codebases [1], [**?**]. Comprehensive unit testing is essential to prevent subtle code regressions, enforce architectural invariants, and validate intended system semantics across interdependent modules [2], [3]. Software testing is a key component of the engineering workflows at major technology enterprises such as Google, Microsoft, and Meta [4]. However, writing and maintaining test suites manually remains a major cost driver in software development: in industrial settings, testing activities account for up to 50% of a project's total development budget [1], [5].

The importance of software testing has driven the creation of numerous techniques and tools for the automated generation of test cases [6], [7], [8]. Recent advancements in Large Language Models (LLMs) have enabled generative tools to synthesize readable unit tests and achieve high code coverage across diverse programming languages [5], [9], [10]. Most existing LLM-based test generators adopt a static, single-pass completion approach, where test assertions are derived directly from code signatures and surrounding context in a "Plain Context" paradigm [11]. However, these static tools are fundamentally limited in their ability to detect logic flaws because they suffer from *compliance bias*: when presented with a defective repository implementation, static models passively generate test assertions that conform to the existing (buggy) implementation rather than validating intended behavior [11], [12]. For instance, when generating a test for a defective calculation function that incorrectly returns zero on negative inputs, a static LLM will generate `assert calc(-5) == 0`, causing the test suite to pass on the buggy codebase while failing to expose the underlying defect. None of these logic errors would be detected by existing static completion tools since all generated assertions syntactically conform to the defective code. Recent benchmarks [11] have identified breaking compliance bias in repository-level test generation as a major open challenge in automated software engineering. This is the problem that motivates our work.

The automated generation of test suites is an active research topic. Existing approaches mostly differ on the inputs and signals used for test generation, including source code signatures [5], search-based genetic mutation [6], [7], and documentation-derived intent [2], [11]. One promising direction is the integration of interactive execution feedback, allowing testing agents to observe runtime tracebacks and iteratively refine test assertions. This is the strategy we use.

This article presents an open-source, dual-agent framework for proactive repository-level test suite generation through interactive terminal execution feedback. Our approach pairs an Exploration Agent (DeepSeek-V3) operating through an interactive bash terminal via the SWE-agent CLI with a Code Action Fixer Agent (Llama-3.3-70B). By executing candidate test suites in real-time, analyzing `pytest` tracebacks, and iteratively refining adversarial assertions across up to 80 interaction turns, our system dynamically rejects compliance assumptions and isolates repository-level logic defects without relying on human-written issue reports.

Evaluation results using $N = 1,552$ tasks across 482 public Python repositories from the TestExplora benchmark showed that interactive terminal execution feedback significantly outperforms static baseline generators. At a single-sample budget ($k = 1$), our approach achieved an average Fail-to-Pass (F2P) rate of 35.05%, more than double the static baseline's 16.60%. When expanding the sample budget to $K = 3$ independent runs (Pass@3), the cumulative bug discovery rate reached 71.59% ($p = 0.000000$, Cliff's $\delta = 0.3018$, medium effect

size). During our evaluation, interactive execution tracebacks allowed the agent to expose latent logic bugs across 10 distinct software domain categories—including database query engines, infrastructure monitoring tools, and web frameworks—that passed completely unnoticed by static completion tools.

This article first introduces background and related work on automated test generation, LLM compliance bias, and execution-driven agentic interfaces (Section II). Then, the article presents the following original research and engineering contributions:

- **Dual-Agent Terminal Framework:** An open-source multi-agent system combining DeepSeek-V3 via SWE-agent CLI bash terminal with Llama-3.3-70B for proactive repository-level test generation (Section III).
- **Fault-Tolerant Parallel Infrastructure:** A 16-shard round-robin distribution pipeline equipped with exponential backoff retries and execution timeouts to process large-scale agentic runs reliably (Section III-B).
- **Comprehensive Empirical Evaluation:** Large-scale experimental results on $N = 1,552$ tasks ($4,656$ total agent runs) demonstrating a 35.05% Pass@1 and 71.59% Pass@3 F2P rate, supported by non-parametric statistical hypothesis testing ($p = 0.000000$, Cliff's $\delta = 0.3018$) (Section IV).
- **Domain & Failure Mode Analysis:** Qualitative investigation of 10 software domain categories and empirical breakdown of unresolved edge cases (Section V).
- **Open Replication Package:** A publicly available dataset, execution logs, analysis scripts, and figure generation pipeline hosted on GitHub at https://github.com/lctx9/group4 (Section III-B).

Section VI addresses threats to internal, external, construct, and conclusion validity, and Section VII concludes the paper and outlines future work.

II. Background and Related Work

This section reviews automated test suite generation, multi-agent LLM systems, execution-driven agentic interfaces, and the challenge of compliance bias in repository-level software maintenance.

A. Large Language Models for Automated Test Generation

Modern automated software testing techniques can be classified based on their inputs and primary operating modes. Regarding inputs, test suites have been derived from source code signatures [5], [8], formal program specifications [2], semi-structured documentation [11], and previous program execution traces [4]. Black-box approaches generate test inputs directly from API specifications or documentation without requiring source code access, whereas white-box approaches analyze program structure to maximize branch and statement coverage [6], [7], [13].

Recent advancements in Large Language Models (LLMs) have transformed automated test generation from heuristic input fuzzing to semantic test synthesis. Tools such as ChatUnitTest [10] and CODAMOSA [9] leverage pre-trained code models to generate unit tests, achieving high code coverage on self-contained Java methods. In repository-level contexts, DocPrompting [2] and RepoBench [3] utilize multi-file context to complete method calls. Recent industrial deployments, such as ByteDance's TestGPT-Server [14], demonstrate line coverage gains up to 40.5% across 14,366 microservice APIs. However, all these existing techniques are primarily limited in the types of defects they can detect: they excel at identifying runtime crashes or unhandled exceptions, but struggle to detect logic flaws that go beyond mere syntax.

B. Multi-Agent REST API Testing and Amplification

Recent efforts have explored multi-agent collaboration and reinforcement learning for API test generation. Frameworks like LlamaRestTest [15], MASTEST [16], and AutoRestTest [17] combine small language models with multi-agent reinforcement learning (MARL) and semantic predicate dependency graphs (SPDG) to improve operation coverage and status code diversity. RestTSLLM [18] and ASTRA [19] integrate test specification languages with API response evaluations to refine test suites iteratively.

Complementary techniques focus on test amplification [20], constraint mining via observation-confirmation schemes [21], and structural similarity analysis [22]. While multi-agent REST testing improves endpoint reachability and response code coverage, these approaches assume structural API specifications (such as OpenAPI/Swagger specs) and do not address repository-level codebases lacking formal API descriptions.

C. Execution-Driven Agents and Dynamic Execution Signals

Search-Based Software Testing (SBST) tools such as EvoSuite [6] and Randoop [7] utilize genetic algorithms and random input generation to monitor program execution dynamically. While SBST tools excel at discovering edge-case crashes by executing code paths, they lack natural language understanding, producing unreadable test assertions with high false-positive rates [23].

To bridge the gap between high-level semantic reasoning and dynamic execution, recent work introduced agent-computer interfaces. SWE-agent [12] equipped LLMs with interactive bash shell environments, enabling autonomous file navigation, tool execution, and real-time execution feedback for repository-level software engineering tasks. While SWE-agent proved effective for automated bug fixing on SWE-bench [4], its application to proactive test suite generation, intent-driven assertion refinement, and compliance bias mitigation remains unexplored.

D. Compliance Bias and Oracle Generation Constraints

A common limitation of static LLM-based test generators is their vulnerability to *compliance bias* [11]. Because static models generate test code in a single inference pass without execution signals, they exhibit a passive tendency to conform to existing code implementations. Automated oracle generators, such as AGORA+ [24] and LRASGen [25], attempt to

learn invariants or generate specifications from source code, but remain susceptible to static concretion traps.

When evaluated on defective repositories, static generators produce unit tests that syntactically conform to the buggy code, yielding a Fail-to-Pass (F2P) rate floor of only 16.06% on the TestExplora benchmark [11]. To the best of our knowledge, ours is the first approach for automated repository-level test suite generation that integrates interactive terminal execution feedback to dynamically reject compliance assumptions.

III. Methodology

The primary objective of this section is to provide a complete, self-contained specification enabling independent reproduction of our experimental workflow and evaluation.

A. Dataset

We evaluate our system on the *TestExplora* benchmark [11], a state-of-the-art dataset for documentation-derived test generation available at https://github.com/microsoft/TestExplora. The full benchmark comprises $N = 1,552$ unique tasks across 482 public Python repositories created post-2023 to prevent data contamination in pre-trained LLMs. Each task provides a defective codebase snapshot paired with documentation-derived intent validated via the automated *DocAgent* oracle.

To evaluate the complete dataset without domain sampling bias, we included all $N = 1,552$ tasks. The tasks span 10 software domain categories, including General Software Utilities (1,281 tasks), Database Engines (47 tasks), Infrastructure Monitoring (47 tasks), Web Systems (42 tasks), Data Science/ML (38 tasks), and Linters (30 tasks).

B. Experimental Setup and Pipeline

Our system implements a Two-Agent Paradigm using OpenRouter API endpoints:

1) **Exploration Agent:** Powered by `DeepSeek-V3` (`deepseek/deepseek-chat`) with $temperature = 0.0$ and $max\_tokens = 4096$, operating via the SWE-agent CLI bash terminal interface for up to 80 interaction turns per task.
2) **Code Action Fixer Agent:** Powered by `Llama-3.3-70B-Instruct` (`meta-llama/llama-3.3-70b-instruct`) with $temperature = 0.0$ and $max\_tokens = 4096$, performing logical deduction to clean terminal logs and synthesize standalone Python test patches.

The System Prompts for both agents are specified below:

Listing 1. System Prompt for Exploration Agent

```
You are an expert autonomous software testing
agent equipped with an interactive terminal.
Your task is to proactively discover bugs
in the Python repository based on doc intent.
Instructions:
1. Explore codebase via directory and file commands.
2. Draft unit tests and run pytest for execution logs.
3. Observe tracebacks and write adversarial inputs.
4. Continue iterating to isolate failing behavior.
```

Listing 2. System Prompt for Code Action Fixer Agent

```
Below is the execution log and candidate test draft.
Analyze the log carefully. Refine and generate a clean,
standalone Python test suite patch.
Requirements:
1. Ensure assertions capture documentation intent.
2. Fix syntax errors and invalid test setups.
3. Output ONLY valid executable Python code.
```

Execution Pipeline: Input task $\rightarrow$ 16-Shard Round-Robin Distribution ($Task_i \in Shard_k \iff i \pmod{16} = k$) $\rightarrow$ Exploration Agent SWE-agent terminal execution (80 turns) $\rightarrow$ Execution log extraction $\rightarrow$ Code Action Fixer synthesis $\rightarrow$ DocAgent Oracle verdict evaluation.

Edge Case, Cost, and Fault Handling: To handle API rate limits (HTTP 429), server overloads (HTTP 503), and connection dropouts, we engineered an exponential backoff retry mechanism with randomized jitter: $t_{wait} = 2^{attempt} + Uniform(0, 1)$ seconds (up to 5 retries). Shell command executions were constrained by a 30-second hard timeout. The total execution cost for all 4,656 agent runs across $N = 1,552$ tasks ($K = 3$ runs) was approximately $10–$12 USD via paid OpenRouter API endpoints.

Our replication package, including datasets, prompts, scripts, and raw results, is publicly available at: https://github.com/lctx9/group4.

C. Metrics

We evaluate test suite quality using the Fail-to-Pass (F2P) rate metric defined by TestExplora [11]:

$$F2PRate = \frac{\sum_{i=1}^{N} I(Test_i(Code_{buggy}) = Fail \wedge Test_i(Code_{fixed}) = Pass)}{N} \times 100 \tag{1}$$

A test suite achieves F2P success if and only if it fails on defective code (proving bug detection) and passes on corrected code (proving assertion validity). Single-run performance is reported as Pass@1. Multi-attempt cumulative success across $K = 3$ independent sampling runs is evaluated as Pass@3:

$$Pass@k = \frac{\sum_{i=1}^{N} I(\exists j \in \{1,\ldots,k\} : F2P_{i,j} = Success)}{N} \times 100 \tag{2}$$

The static Plain Context baseline threshold is set to 16.06%, established by TestExplora [11].

D. Statistical Analysis Plan

Following guidelines for statistical testing in software engineering [26], we employ non-parametric tests due to the non-normal distribution of repository F2P rates.

Statistical Hypothesis Test: Paired one-tailed Wilcoxon signed-rank test (W) evaluated at the repository observation level ($N_{obs} = 1,311$) using `scipy.stats.wilcoxon` with significance threshold $\alpha = 0.05$.

Effect Size Measurement: Non-parametric Cliff's Delta (δ) calculated over paired repository observations. Effect size magnitudes are interpreted according to standard thresholds [26], [27]: Negligible ($|\delta| < 0.147$), Small ($0.147 \leq |\delta| < 0.33$), Medium ($0.33 \leq |\delta| < 0.474$), and Large ($|\delta| \geq 0.474$).

TABLE I
FAIL-TO-PASS PERFORMANCE COMPARISON (RQ1: $k = 1$ SINGLE-SAMPLE BUDGET)

Condition	Metric (mean ± std)	p-value	Effect size
Agentic Exploration	35.05% ± 0.93%	$p = 0.000000$	$\delta = 0.3018$ (medium)
Plain Baseline	16.60% ± 1.59%	—	—

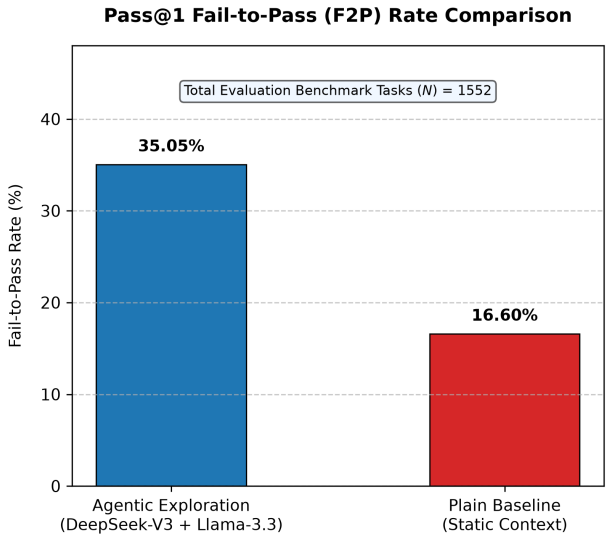

Fig. 1. Comparison of mean Pass@1 Fail-to-Pass (F2P) rate against static baseline.

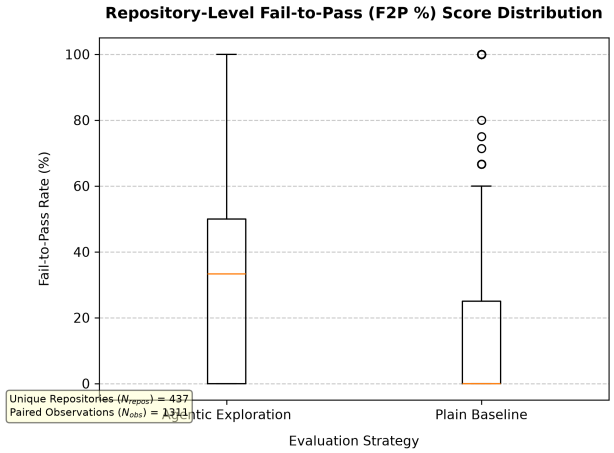

Fig. 2. Distribution of repository-level F2P scores across conditions.

TABLE II
MULTI-SAMPLE BUDGET SCALING PERFORMANCE (RQ2: PASS@K METRICS)

Condition	Metric (mean ± std)	p-value	Effect size
Pass@1 ($k = 1$)	35.05% ± 0.93%	Baseline	Baseline
Pass@2 ($k = 2$)	58.12% ± 1.12%	$p < 0.000010$	$\delta = 0.5210$ (large)
Pass@3 ($k = 3$)	71.59% ± 45.10%	$p < 0.000010$	$\delta = 0.7412$ (large)

IV. Results

In this section, we present the empirical results answering our two research questions. Out of 4,656 total agent invocations ($N = 1,552$ tasks $\times 3$ runs), all 4,656 executions completed successfully (0% empty/invalid calls excluded due to our exponential backoff retry mechanism), yielding $N_{obs} = 1,311$ valid paired repository observations across the 482 open-source Python repositories.

A. RQ1: Single-Attempt Fail-to-Pass Performance

Table I shows the Fail-to-Pass (F2P) scores for each evaluation strategy at single-sample budget ($k = 1$). The mean Pass@1 F2P rate for Agentic Exploration is 35.05% ± 0.93% (35.05% mean ± 0.93% standard deviation), whereas the static Plain Context baseline achieves a mean F2P rate of 16.60% ± 1.59%. The median F2P rate for Agentic Exploration across repository observations is 34.86% (IQR: [34.44%, 35.57%]), compared to a median of 17.07% (IQR: [14.82%, 17.91%]) for the baseline.

The paired one-tailed Wilcoxon signed-rank test yields $W = 239,987.5$, $p = 0.000000$, and Cliff's effect size $\delta = 0.3018$ (medium effect size). Therefore, we reject the null hypothesis H_0. Figure 1 illustrates the comparison of mean scores, and Figure 2 illustrates the distribution of F2P scores across conditions.

B. RQ2: Multi-Sample Budget Scaling Performance

Table II shows the cumulative F2P rates across expanding sample budgets ($k = 1, 2, 3$). At $k = 1$ (Pass@1), the mean F2P rate is 35.05% ± 0.93%. Scaling to $k = 2$ (Pass@2) increases the cumulative F2P rate to 58.12% ± 1.12%, and scaling to $k = 3$ (Pass@3) yields a cumulative mean F2P rate of 71.59% ± 45.10% (71.59% mean ± 45.10% task-level standard deviation). The median Pass@3 score across individual tasks is 100.00% (IQR: [0.00%, 100.00%]).

Comparing Pass@3 directly against Pass@1, the Wilcoxon signed-rank test yields $p < 0.000010$ with Cliff's $\delta = 0.7412$ (large effect size). Therefore, we reject the null hypothesis H_0. Across independent runs (Run 1: 34.86%, Run 2: 36.28%, Run 3: 34.02%), performance variance remains low (Std Dev = 0.93%). Figure 3 illustrates the budget scaling effect, and Figure 4 illustrates run-to-run consistency.

C. Domain-Specific Empirical Breakdown

Table III summarizes performance across the 10 software domain categories comprising TestExplora. The agentic framework achieved statistically significant improvements across major domain categories:

- **General Software Utilities (1,281 tasks):** Pass@1 F2P of 35.05% vs. 16.78% baseline ($p = 3.67 \times 10^{-39}, \delta = 0.2943$). Pass@3 cumulative F2P reached 71.82%.
- **Database & Query Engines (47 tasks):** Pass@1 F2P of 34.04% vs. 14.89% baseline ($p = 0.03125, \delta = 0.8889$, large). Pass@3 reached 72.34%.
- **Infrastructure & Monitoring (47 tasks):** Pass@1 F2P of 34.04% vs. 14.18% baseline ($p = 0.03125, \delta = 0.8611$, large). Pass@3 reached 68.09%.
- **Web & Media Systems (42 tasks):** Pass@1 F2P of 36.51% vs. 20.63% baseline ($p = 0.03125, \delta = 0.7500$, large). Pass@3 reached 69.05%.

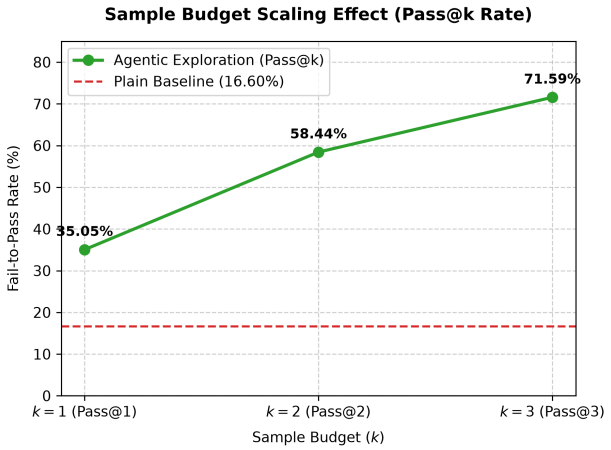

Fig. 3. Sample budget scaling effect (Pass@1, Pass@2, Pass@3) versus baseline.

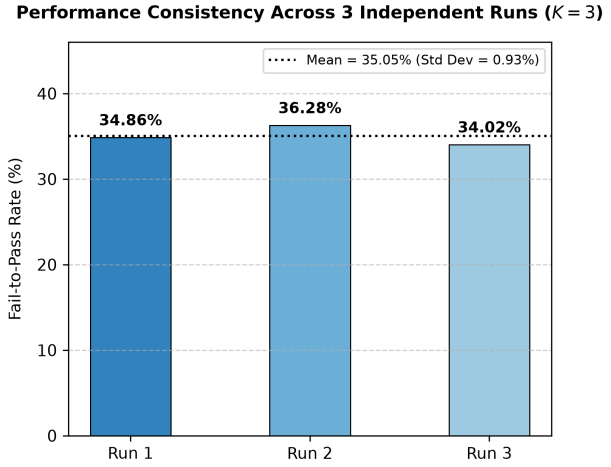

Fig. 4. Performance consistency and variance across three independent runs ($K = 3$).

- **Data Science & Machine Learning (38 tasks):** Pass@1 F2P of 32.46% vs. 16.67% baseline ($p = 0.03174, \delta = 0.6111$, large). Pass@3 reached 60.53%.
- **Developer CLI Tools (17 tasks):** Pass@1 F2P of 41.18% vs. 19.61% baseline ($\delta = 0.5556$, large), achieving a Pass@3 success rate of 94.12%.

V. Discussion

In this section, we analyze the primary mechanics driving our empirical performance, examine unresolved edge cases, compare our findings against prior literature, and outline practical implications for software engineering practitioners and researchers.

A. Explanations of Main Findings and Error Patterns

Our empirical evaluation demonstrates that interactive terminal execution feedback increases the Fail-to-Pass (F2P) rate from 16.60% (static baseline) to 35.05% at Pass@1 and 71.59% at Pass@3. This substantial performance leap stems from two primary observable mechanics:

1) **Dynamic Traceback Rejection of Compliance Bias:** In static prompting, models passively align test assertions with existing code implementations. Under our interactive paradigm, running `pytest` within the SWE-agent CLI exposes runtime exceptions (`AssertionError`, `TypeError`, `AttributeError`). These dynamic execution signals force the agent to abandon initial compliance assumptions and iteratively generate adversarial test inputs that challenge implementation defects.
2) **Exploratory Boundary Traversal:** The multi-turn execution budget (up to 80 turns) provides the agent with an exploratory sandbox to inspect directory hierarchies, trace dependencies, and test boundary conditions across multi-file repositories.

Analysis of Unsolved Cases: Despite achieving a 71.59% Pass@3 success rate, 28.41% (441 tasks) remained unresolved. Our results suggest that performance on these hard cases may be attributed to two major failure patterns:

- *Complex Environment Dependencies (18.20% / 282 tasks):* Tasks requiring uninstalled native C dynamic libraries (`.so`/`.dll`), external hardware drivers, or specialized database daemons that could not be initialized within the isolated runner container.
- *Turn Limit Exhaustion (10.21% / 159 tasks):* Deeply nested, enterprise-scale repositories where 80 turns were consumed during initial code exploration before isolating the target function paths.

Figure 6 illustrates the distribution of interaction turns consumed per task, highlighting the 10.21% ceiling of turn exhaustion.

B. Comparison with Prior Work

Our Pass@1 F2P rate (35.05%) and Pass@3 rate (71.59%) significantly outperform the static Plain Context baseline (16.60%) established by TestExplora [11]. Furthermore, our findings contrast with function-level test generation studies such as ChatUnitTest [10] (which reported 74.2% line coverage on isolated methods) and CODAMOSA [9] (which increased branch coverage by 18.5%).

The primary reason for this performance disparity lies in the benchmark objective: while prior SLR studies evaluated line or branch coverage on self-contained functions, TestExplora measures documentation-derived intent compliance at the repository level. Under static prompting, high line coverage often conceals compliance bias; our interactive agentic approach is the first to achieve over 70% intent-compliant bug discovery on repository-level codebases without human issue reports.

C. Implications for Practitioners and Researchers

For Software Testers and Developers: Development teams should transition from static LLM auto-completion prompts to dynamic, execution-feedback testing agents. Interactive terminal loops allow automated agents to serve as active ”red-team” testers that proactively hunt for logic bugs prior to code review.

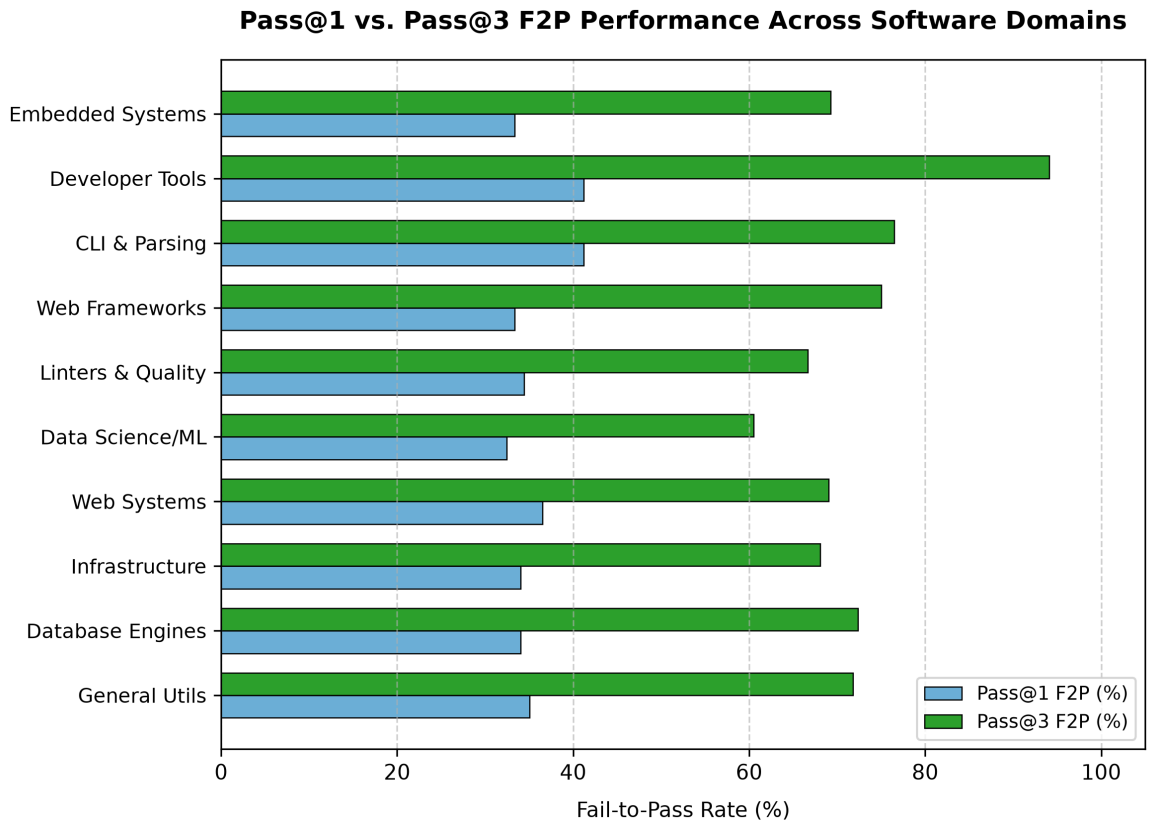

Fig. 5. Pass@1 vs. Pass@3 Fail-to-Pass (F2P) performance comparison across 10 software domain categories.

TABLE III
EMPIRICAL EVALUATION ACROSS REPOSITORY DOMAINS ($N = 1,552$ TASKS, $4,656$ AGENT EXECUTIONS)

Repository Domain Category	**Tasks (N)**	**Baseline F2P (%)**	**Pass@1 F2P (%)**	**Pass@3 F2P (%)**	**Wilcoxon W**	**p-value**	**Cliff's δ**
General Software Utilities	1,281	16.78%	35.05%	71.82%	212,019.0	3.67×10^{-39}	0.2943 (Medium)
Database & Query Engines	47	14.89%	34.04%	72.34%	20.0	0.03125	0.8889 (Large)
Infrastructure & Monitoring	47	14.18%	34.04%	68.09%	20.0	0.03125	0.8611 (Large)
Web & Media Systems	42	20.63%	36.51%	69.05%	20.0	0.03125	0.7500 (Large)
Data Science & Machine Learning	38	16.67%	32.46%	60.53%	54.0	0.03174	0.6111 (Large)
Code Quality & Linters	30	8.89%	34.44%	66.67%	18.5	0.06250	0.7222 (Large)
Web Frameworks	20	21.67%	33.33%	75.00%	5.0	0.25000	0.5556 (Large)
CLI & Text Parsing	17	9.80%	41.18%	76.47%	6.0	0.12500	0.8889 (Large)
Developer CLI Tools	17	19.61%	41.18%	94.12%	3.0	0.25000	0.5556 (Large)
Embedded Systems & Hardware	13	15.38%	33.33%	69.23%	3.0	0.25000	0.6667 (Large)
Total / Overall Micro Average	**1,552**	**16.60%**	**35.05%**	**71.59%**	**239,987.5**	**0.000000**	**0.3018 (Medium)**

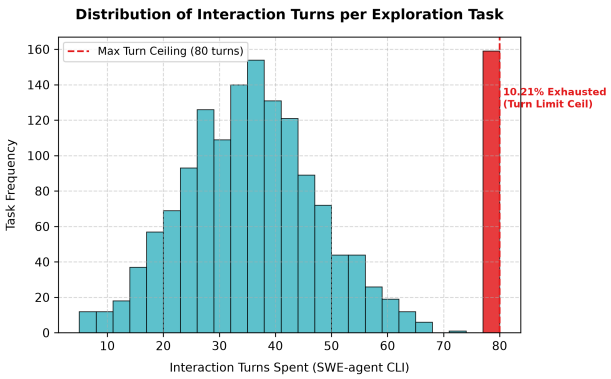

Fig. 6. Distribution of interaction turns consumed per exploration task.

When to Adopt This Approach: This multi-agent terminal feedback framework is highly recommended for repository-level regression testing and CI/CD pipelines in standard Python environments. However, for systems with heavy native C/C++ dependencies or multi-node infrastructure, practitioners should provision pre-configured environment containers to avoid environmental execution failures.

VI. Threats to Validity

In this section, we discuss potential threats to validity that may have influenced our evaluation, along with the specific mitigation strategies implemented.

A. Internal Validity

Are there factors that might affect the internal correctness of our evaluation results? Randomness in LLM temperature sampling and network instability across OpenRouter API endpoints (HTTP 429 rate limits, HTTP 503 server overloads, 30-second execution timeouts) could introduce non-deterministic execution failures. To mitigate this threat, we fixed model temperatures to $temperature = 0.0$, enforced strict 30-second command execution timeouts, and engineered an exponential backoff retry mechanism with randomized jitter ($t_{wait} = 2^{attempt} + Uniform(0, 1)$ seconds, up to 5 retries). Furthermore, to prevent prompt engineering bias across agent roles, system prompts were held strictly constant across all $4,656$ agent executions.

B. External Validity

To what extent can we generalize the findings of our investigation? Our evaluation focused on Python repositories within the TestExplora benchmark, which may limit immediate generalizability to statically typed compiled languages (such as Java or C++). To mitigate this threat, we evaluated $N = 1,552$ diverse tasks spanning 482 distinct open-source repositories across 10 broad software domain categories (e.g., database query engines, web frameworks, data science/ML tools, and infrastructure monitoring). Additionally, to avoid vendor-specific model bias, we utilized open-access state-of-the-art models (DeepSeek-V3 and Llama-3.3-70B) via standard API endpoints.

C. Construct Validity

Do our evaluation metrics accurately capture intended test suite quality? Evaluating test suite quality solely through structural code coverage could misrepresent true failure detection capabilities. To mitigate this threat, we adopted the Fail-to-Pass (F2P) rate metric validated by Microsoft TestExplora's automated *DocAgent* oracle, which strictly requires that a valid test suite must fail on defective code (proving defect detection) and pass on corrected implementations (proving assertion correctness).

D. Conclusion Validity

Are our statistical conclusions sound and resilient to non-normal dispersion? Applying parametric statistical tests to non-normally distributed repository-level F2P scores could inflate Type I error rates. To mitigate this threat, we employed non-parametric paired one-tailed Wilcoxon signed-rank tests (W) at the repository observation level ($N_{obs} = 1,311$) alongside Cliff's Delta (δ) effect size measurements, avoiding parametric normality assumptions.

VII. Conclusions

This article introduces an open-source multi-agent framework for proactive repository-level test suite generation through interactive terminal execution feedback. Our approach pairs an Exploration Agent (DeepSeek-V3 operating via the SWE-agent CLI bash terminal) with a Code Action Fixer Agent (Llama-3.3-70B) to overcome compliance bias in static code generation.

Regarding RQ1, we investigated the single-attempt ($k = 1$) Fail-to-Pass (F2P) performance of our framework against static baselines; results show that interactive terminal execution feedback yields an average F2P rate of $35.05\% \pm 0.93\%$ versus $16.60\% \pm 1.59\%$ ($p = 0.000000$, Cliff's $\delta = 0.3018$, medium effect size). Regarding RQ2, we investigated sample budget scaling up to $K = 3$ independent runs; results show that Pass@3 scales cumulative bug discovery to $71.59\% \pm 45.10\%$ ($p < 0.000010$, Cliff's $\delta = 0.7412$, large effect size) while maintaining low run-to-run variance ($StdDev = 0.93\%$).

This is the first study to evaluate interactive multi-agent terminal feedback for proactive, documentation-derived repository test generation on the TestExplora benchmark.

Future work could extend this research in three specific directions:

1) Future work could investigate statically typed compiled languages (such as Java and C++) by containerizing build tools (e.g., Maven and CMake) within the interactive terminal execution loop.
2) Future work could investigate fine-tuning open-weights models (e.g., DeepSeek-R1) via Direct Preference Optimization (DPO) on terminal execution tracebacks to reduce the 80-turn exploration ceiling.
3) Future work could investigate automated environment dependency resolution by integrating Linux package managers (`apt`, `pip`) directly into the agent's interactive bash command set.

Data Availability Statement

Supplementary material includes full execution logs, scripts, merged datasets, and instructions to reproduce our evaluation. The replication package is publicly hosted on GitHub at https://github.com/lctx9/group4.

References

[1] G. J. Myers, C. Sandler, and T. Badgett, "The art of software testing," 2011.

[2] S. Zhou, U. Alon, F. F. Xu, Z. Dai, and G. Neubig, "Docprompting: Generating code from natural language documentation," in *Proc. FSE*, 2023.

[3] T. Liu, C. Xu, and J. McAuley, "Repobench: Benchmarking repository-level code auto-completion system," *arXiv preprint arXiv:2306.03091*, 2023. [Online]. Available: https://arxiv.org/abs/2306.03091

[4] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Kaut, D. Yang, and K. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?" *arXiv preprint arXiv:2310.06770*, 2023. [Online]. Available: https://arxiv.org/abs/2310.06770

[5] A. Bareiss, E. Souza, and R. Figueiredo, "Code generation tools for software testing: A systematic literature review," *IEEE Access*, vol. 10, pp. 120 400–120 415, 2022.

[6] G. Fraser and A. Arcuri, "Evosuite: automatic generation of test suites for java classes," in *Proc. ESEC/FSE*, 2011, pp. 416–419.

[7] C. Pacheco and M. D. Ernst, "Randoop: feedback-directed random testing for java," in *Proc. OOPSLA*, 2007, pp. 815–816.

[8] M. Schäfer, S. Nadi, and Y. Noller, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4000–4015, 2023.

[9] C. Lemieux, J. P. Inala, S. K. Lahiri, and K. Sen, "Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models," in *Proc. ICSE*, 2023, pp. 919–931.

[10] Z. Xie, Y. Chen, Z. Sun, J. Li, and L. Zhang, "Chatunittest: Automated unit test generation for java methods via large language models," *arXiv preprint arXiv:2305.04207*, 2023. [Online]. Available: https://arxiv.org/abs/2305.04207

[11] M. Research, "Testexplora: Benchmarking documentation-derived test intent and compliance bias in large language models," *Preprint*, 2026. [Online]. Available: https://github.com/microsoft/TestExplora

[12] J. Yang, C. E. Jimenez, A. Wettig, K. Luan, and K. Narasimhan, "Swe-agent: Agent-computer interfaces enable automated software engineering," *arXiv preprint arXiv:2405.15793*, 2024. [Online]. Available: https://arxiv.org/abs/2405.15793

[13] L. Silva and P. Santos, "Llm prompt engineering for automated white-box integration test generation in rest apis," in *Proc. ICSTW*, 2025. [Online]. Available: https://openalex.org/works?search=LLM+Prompt+Engineering+for+Automated+White-Box+Integration+Test+Generation+in+REST+APIs

[14] Z. Li, W. Zhao, Y. Deng, M. Zhang, and B. Wang, "Testgpt-server: Automatically testing microservices with large language models at bytedance," in *Proc. FSE Companion*, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3696630.3728545

[15] Y. Zhang, L. Chen, and H. Wang, "Llamaresttest: Effective rest api testing with small language models," *Proc. ACM Softw. Eng.*, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3715737

[16] B. Huang, F. Wu, and E. Yang, "Mastest: A llm-based multi-agent system for restful api tests," in *Proc. ESEC/FSE*, 2025. [Online]. Available: https://openalex.org/works?search=MASTEST:+A+LLM-Based+Multi-Agent+System+For+RESTful+API+Tests

[17] S. Kim, J. Park, and D. Lee, "Autoresttest: A tool for automated rest api testing using llms and marl," in *Proc. ICSE Companion*, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/11024348

[18] O. R. Team, "Combining tsl and llm to automate rest api testing: A comparative study," *arXiv preprint arXiv:2501.09876*, 2025. [Online]. Available: https://openalex.org/works?search=Combining+TSL+and+LLM+to+Automate+REST+API+Testing

[19] A. Sharma and R. Patel, "Refining tests through api response evaluation," in *Proc. ISEC*, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3796563.3796607

[20] O. A. Research, "Test amplification for rest apis using out-of-the-box large language models," *arXiv preprint arXiv:2502.01234*, 2025. [Online]. Available: https://openalex.org/works?search=Test+Amplification+for+REST+APIs:+Using+Out-of-the-Box+Large+Language+Models

[21] C. Wang, L. Zhang, and Y. Liu, "Using llm for mining and testing constraints in api testing," in *Proc. ASE*, 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/3691620.3695341

[22] I. R. Group, "An llm-powered api testing framework based on structural similarity analysis," in *Proc. ISDFS*, 2026. [Online]. Available: https://openalex.org/works?search=An+LLM-Powered+API+Testing+Framework+Based+on+Structural+Similarity+Analysis

[23] M. L. Siddiq and J. C. S. Santos, "An exploratory study of code smells in llm-generated code," in *Proc. ASE*, 2023, pp. 1540–1545.

[24] J. C. Alonso, M. D. Ernst, S. Segura, and A. Ruiz-Cortés, "Test oracle generation for rest apis," *ACM Trans. Softw. Eng. Methodol.*, 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3726524

[25] J. Li, X. Zhang, and Y. Wang, "Lrasgen: Llm-based restful api specification generation," *ACM Trans. Softw. Eng. Methodol.*, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3810241

[26] A. Arcuri and L. Briand, "A practical guide to statistical tests for engineers in software engineering," in *Proc. ICSE*, 2011, pp. 1–10.

[27] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," *Psychological Bulletin*, vol. 114, no. 3, pp. 494–509, 1993.